

PROFESSIONAL PYTHON NOTES

1. OPERATORS

Operation

An **operation** is a task or action performed on values based on the operator used.

Operand

An **operand** is the value on which an operation is performed.

Operator

An **operator** is a special symbol or keyword that provides inbuilt functionality to perform an operation.

Expression

An **expression** is a statement that performs an operation.

Example

8 * 4

Term	Meaning
Operation	Multiplication
Operands	8, 4
Operator	*
Expression	8 * 4

Types of Operators

There are **7 types of operators**:

1. Arithmetic Operator
2. Relational / Comparison Operator
3. Logical Operator
4. Bitwise Operator
5. Assignment Operator
6. Membership Operator
7. Identity Operator

2. ARITHMETIC OPERATORS

Arithmetic operators are used to perform mathematical operations on numbers.

Operators

- + Addition
- Subtraction

* Multiplication
** Exponentiation
// Floor Division
/ Float Division
% Modulo

The source introduces these operators and their examples on pages 1–3.

2.1 Addition +

When used with a minimum of two numbers, + performs addition.

When used with a single number, it represents a positive number.

Example

```
print(3 + 4)          # addition
print(39.6 + 12.4 + 4 + 8)  # addition
print(+113)           # positive number
print(+12.5)          # positive number
```

2.2 Subtraction -

When used with a minimum of two numbers, - performs subtraction.

When used with a single number, it represents a negative number.

Example

```
print(10 - 3)         # subtraction
print(12.3 - 3.4 - 5) # subtraction
print(-12.45)         # negative number
print(-23)            # negative number
```

2.3 Multiplication *

When used with a minimum of two operands, * performs multiplication.

Example

```
print(3 * 4)          # multiplication
print(12.4 * 3.6)     # multiplication
```

A multiplication operation requires operands on both sides.

2.4 Exponentiation **

The exponentiation operator returns the value obtained by raising the first number to the power of the second number.

Example

```
print(2 ** 3)
print(3 ** 2)
print(2.2 ** 2)
print(5 ** 1.5)
print(1 ** 0)
```

Important

`2 ** 3 = 8`

Here:

- `2` → base
 - `3` → exponent
 - `8` → result
-

3. DIVISION OPERATORS

Python provides different operators for division.

3.1 Floor Division //

It returns the **non-decimal quotient**.

Example

```
print(15 // 3) # 5
print(17 // 3) # 5
print(25 // 8) # 3
```

Remember

`//` → Floor division

3.2 Float Division /

It returns the **decimal quotient**.

Example

```
print(15 / 3) # 5.0
print(11 / 2) # 5.5
print(25 / 8) # 3.125
```

Difference

`15 // 3` → 5

$15 / 3 \rightarrow 5.0$

3.3 Modulo %

The modulo operator returns the **remainder** of a division operation.

Example

```
print(15 % 3) # 0
```

```
print(17 % 3) # 2
```

```
print(25 % 8) # 1
```

```
print(3 % 84) # 3
```

```
print(15 % 2) # 1
```

Important

$17 \div 3$

Quotient = 5

Remainder = 2

Therefore:

$17 // 3 = 5$

$17 \% 3 = 2$

4. COMPARISON / RELATIONAL OPERATORS

Comparison operators are used to compare values or determine the relationship between values.

They return a **Boolean result** such as:

True

False

Operators

< Less than

> Greater than

== Equal to

<= Less than or equal to

>= Greater than or equal to

!= Not equal to

They can be used with:

1. Direct values or variables
2. Boolean values
3. Expressions that return values

The source introduces comparison operators on page 4.

Example

```
a = 10
```

```
b = 20
```

```
print(a < b)
```

```
print(a > b)
```

```
print(a == b)
```

```
print(a <= b)
```

```
print(a >= b)
```

```
print(a != b)
```

5. LOGICAL OPERATORS

Logical operators are used to **combine and check multiple Boolean conditions**.

Operators

and

or

not

The source notes that values other than 0, None, and an empty string are treated as truthy in Boolean contexts.

5.1 Logical and

The and operator returns a truthy result only when the required conditions are true.

Truth Table

C1	C2	Result
-----------	-----------	---------------

True	True	True
------	------	------

True	False	False
------	-------	-------

False	True	False
-------	------	-------

False	False	False
-------	-------	-------

Same examples from the notes

```
print(((65 + 3) <= (39)) and 3 and True)
print(10 and -5 and 3)
print((106 > 110) and (-1 <= -3) and (7))
print(-6 and -3 and 0)
print(None and 0 and "")
```

5.2 Logical or

The or operator returns a truthy result when at least one condition is true.

Truth Table

C1	C2	Result
----	----	--------

True	True	True
------	------	------

False	True	True
-------	------	------

True	False	True
------	-------	------

False	False	False
-------	-------	-------

Examples

```
print(True or "")
print("" and 5) or False
print((106 > 110) or (-1 <= -3) or (7))
print(-6 or -3 or 0)
print(None or 0 or "")
```

5.3 Logical not

The not operator requires **one operand**.

It reverses the Boolean meaning of the operand.

Examples

```
print(not ((10 + 5) > (15 + 2)) or (64 and 92 and -3))
print(not(""))
print(not(None))
print(not(True or True))
print(not(-3))
print(not(" "))
print(not(9))
```

Important Boolean Values

The source specifically discusses:

True

False

None

These are special built-in values/keywords.

6. BITWISE OPERATORS

Bitwise operators operate at the bit level.

The source lists:

& Bitwise AND

| Bitwise OR

~ Bitwise NOT / Complement

^ Bitwise XOR

<< Left Shift

>> Right Shift

>>> Unsigned Right Shift

Important

The ~ operator requires only a single operand.

Example:

```
a = 5
```

```
print(~a)
```

7. ASSIGNMENT OPERATOR

The assignment operator is:

=

It is used to:

- Assign a value
- Initialize a variable
- Re-assign a variable
- Copy a value from one variable to another

Syntax

```
var_name = value
```

```
var_name = initialized_var_name
```

```
var_name = expression
```

Examples

```
a = 10
```

```
b = a
```

```
c = 20 + 5
```

The source defines assignment and variables together on page 6.

8. VARIABLES

A **variable** is a named memory location that holds a value.

It can also be understood as a container that holds a value.

Two important steps

Every variable should follow:

1. Initialization
2. Utilization

Example

```
a = 10    # initialization
```

```
print(a)  # utilization
```

A variable should be initialized before it is utilized.

9. MEMORY, id() AND type()

The notes discuss how variables refer to values in memory.

If the same value is assigned to multiple variables, the Python virtual machine may allow the variables to refer to the same object.

Example

```
a = 15
```

```
b = 10 + 5
```

```
c = 20 - 5
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

The notes also introduce:

id()

id() returns the identity associated with an object during execution.

```
print(id(a))
```

type()

type() returns the datatype/class of an object.

```
print(type(a))
```

The source demonstrates these concepts with variables a, b, c, d, i, and k.

10. COMPOUND ASSIGNMENT OPERATORS

A combination of an arithmetic/bitwise operator with assignment is called a **compound statement/operator**.

Arithmetic + Assignment

+=

-=

*=

**=

//=

/=

%=

Bitwise + Assignment

&=

|=

^=

<<=

>>=

>>>=

Example

```
i = i + (j * 5)
```

can be written as:

```
i += (j * 5)
```

Another example:

```
i = i - 5
```

can be written as:

```
i -= 5
```

Rule

The variable being updated on the left side must also participate in the operation on the right side.

11. MEMBERSHIP OPERATORS

Membership operators are:

in

not in

They return a Boolean value.

They are used to check whether a value exists inside a group of elements.

Syntax

value in iterable

Iterable Objects

Examples include:

List

Tuple

Set

String

Dictionary

Example

```
l1 = [10, 20, 30]
```

```
print(20 in l1)
```

```
print(50 in l1)
```

```
print(20 not in l1)
```

The source explains membership operators on page 9.

12. IDENTITY OPERATORS

Identity operators compare the **identity of two objects**.

They return a Boolean result.

Common identity operators are:

is

is not

They check whether two references refer to the same object.

The source describes identity operators as checking the IDs of two values.

13. CONTROL FLOW STATEMENTS

Control flow statements control the **execution flow of a program**.

The source divides them into three major categories: conditional, looping and jumping statements.

Types

1. Conditional Statements

- Simple if
- if-else
- elif ladder
- Shorthand if

2. Looping Statements

- for
- while

3. Jumping Statements

- break
- continue
- return
- pass

14. CONDITIONAL STATEMENTS

Conditional statements execute particular code depending on whether a condition is true or false.

14.1 Simple if

Syntax

```
if bool_condition:
```

```
    # logic
```

```
# remaining statements
```

Example

```
num = int(input("enter a number: "))
```

```
if num > 0:
```

```
print(num, "is positive")
```

```
print("program is executed")
```

Working

If:

```
num > 0
```

is True, the message is displayed.

Otherwise, the if block is skipped.

15. if-else

if-else provides two possible execution paths.

Syntax

```
if bool_condition:
```

```
    # logic
```

```
else:
```

```
    # alternate logic
```

```
# remaining statements
```

Example from the source

```
num = int(input("enter a number: "))
```

```
if num > 0:
```

```
    print(num, "is positive")
```

```
else:
```

```
    print(num, "is not positive")
```

```
print("program is executed")
```

The source includes this example and the corresponding flow diagram on page 12.

16. elif LADDER

An elif ladder is used when there are multiple conditions to check.

Syntax

```
if bool_cond1:
```

```
    # logic1
```

```
elif bool_cond2:
```

```
    # logic2
```

```
elif bool_cond3:
```

```
    # logic3
```

```
else:
```

```
    # alternate logic
```

```
# remaining statement
```

The source also mentions a **work-calendar example** for the elif ladder.

17. SHORTHAND if

A conditional expression can be written in a short form.

Example

```
n = int(input("enter a number: "))
```

```
res = "positive number" if n > 0 else "not a positive number"
```

```
print(res)
```

Another example from the notes:

```
n = int(input("enter a number: "))
```

```
res = "positive number" if n > 0 else "negative number" if n < 0 else "neutral"
```

```
print(res)
```

This is useful when the logic is simple.

18. DATA TYPES

A datatype specifies the type of value stored in a variable.

The source notes that Python does not use fixed data sizes in the same manner as some statically typed languages and lists the following types.

Main Data Types

Datatype	Python Type	Example
Integer	int	10
Decimal	float	10.5
String	str	"Rama"
List	list	[10,20,"Rama",30]
Tuple	tuple	(10,20,"Rama",30)
Set	set	{10,20,30}
Frozen Set	frozenset	frozenset((10,20,30))
Dictionary	dict	{"name":"raju"}

Integer

a = 10

Float

a = 10.5

String

A string is a collection of characters.

a = "Rama"

List

a = [10, 20, "Rama", 30]

A list is mutable.

Tuple

a = (10, 20, "Rama", 30)

A tuple is immutable.

Set

a = {10, 20, 30}

A set is an unordered collection and is mutable.

Dictionary

A dictionary stores data as **key-value pairs**.

a = {"name": "raju"}

19. CODING RULES / OPTIMIZATION

The source gives several coding rules.

Rule 1

Code should be dynamic.

Rule 2

Name members of the code based on the values stored or the action performed.

Rule 3

Try to keep specific logic within the scope of a function.

Rule 4

Try to keep code efficient and optimized.

Optimization

Optimization means:

Reducing the number of lines of code without changing the actual meaning.

The source also recommends avoiding unnecessary use of built-in functions or methods until they are required.

20. range() FUNCTION

range() is a predefined function used to generate a sequence of values that can be accessed individually, commonly through a for loop.

Syntax

```
range(start, actual_end, step)
```

21. FOR LOOP WITH range()

Syntax

```
for var_name in range(start, actual_end, step):
```

```
    # logic
```

```
# remaining statements
```

Increment Loop

Example from the notes

WAL to display from 1 to n (included) in increment order

```
n = int(input("enter a number: "))
```

```
for i in range(1, n + 1, 1):
```

```
    print(i, end=" ")
```

```
print("\nlast updated value i :", i)
```

If input is:

12

Output

1 2 3 4 5 6 7 8 9 10 11 12

last updated value i : 12

The source demonstrates this example and traces the loop values from 1 through 12.

22. FOR LOOP TRACING

Example:

```
for i in range(1, 5):
```

```
    print(i, end=" ")
```

Tracing

$i = 1 \rightarrow 1 < 5 \rightarrow \text{print } 1$

$i = 2 \rightarrow 2 < 5 \rightarrow \text{print } 2$

$i = 3 \rightarrow 3 < 5 \rightarrow \text{print } 3$

$i = 4 \rightarrow 4 < 5 \rightarrow \text{print } 4$

$i = 5 \rightarrow 5 < 5 \rightarrow \text{False} \rightarrow \text{exit}$

Output

1 2 3 4

23. DECREMENT LOOP

A decrementing loop uses a negative step.

General form

```
for var_name in range(start, end, negative_step):
```

```
    # logic
```

Example

To display values in decreasing order with a difference of 2:

```
for i in range(7, 0, -2):
```

```
    print(i, end=" ")
```


Output:

7 5 3 1

The source introduces decrement loops and a question to display n to 1 in decrementing order with a difference of 2.

24. FOR LOOP WITHOUT range()

A for loop can directly iterate through an iterable object or group of elements.

Syntax

```
for var_name in group_of_elements:  
    # logic
```

```
# remaining statements
```

Example

```
l1 = [10, 23, "ab", [1, 2], 32]
```

```
for val in l1:  
    print(val, end=" ")
```

Output:

10 23 ab [1, 2] 32

The source demonstrates iteration over lists, strings and other iterable values on pages 16–17.

25. ITERABLE

An **iterable** is an object whose elements can be accessed one by one during iteration.

Examples from the notes include:

List

Tuple

Set

String

Dictionary

Example

```
s1 = "abc"
```

```
for val in s1:  
    print(val, end=" ")
```

Output:

a b c

The cursor moves through the elements in the iterable.

26. LIST INDEXING WITH for

The source uses the following list:

```
l1 = [11, -2, 3, 15]
```

and demonstrates accessing elements using indexes.

Example

```
l1 = [11, -2, 3, 15]
```

```
for i in range(0, len(l1)):
```

```
    print(l1[i] * 10)
```

Index values

Index 0 → 11

Index 1 → -2

Index 2 → 3

Index 3 → 15

Tracing

i = 0 → print(l1[0] * 10)

i = 1 → print(l1[1] * 10)

i = 2 → print(l1[2] * 10)

i = 3 → print(l1[3] * 10)

27. WHILE LOOP

A while loop repeatedly executes a block of code while a Boolean condition remains true.

Syntax

```
# initialize the variable
```

```
while bool_condition:
```

```
    # logic
```

```
    # update the looping variable
```

```
# remaining statements
```

The source introduces the while loop on pages 18–20.

28. TYPES OF VARIABLES IN A LOOP

The notes identify three varieties:

1. **Looping variable**
 2. **Conditional variable**
 3. **Looping + conditional variable**
-

29. WHILE LOOP EXAMPLE

WAL to display natural numbers up to n

```
n = int(input("enter a number: "))
```

```
i = 3
```

```
while i <= n:
```

```
    print(i, end=" ")
```

```
    i = i + 1
```

```
print("\nlast updated value i:", i)
```

Tracing for n = 5

```
i = 3
```

```
3 <= 5 → True → print 3
```

```
i = 3 + 1 → 4
```

```
4 <= 5 → True → print 4
```

```
i = 4 + 1 → 5
```

```
5 <= 5 → True → print 5
```

```
i = 5 + 1 → 6
```

```
6 <= 5 → False
```

```
Exit loop
```

30. WHILE LOOP WITH DECREMENT

The notes also demonstrate a decrementing while loop.

Example concept:

$n = 8$

$8 \rightarrow 7 \rightarrow 5 \rightarrow 2 \rightarrow \dots$

The source traces changing values and the difference variable during the loop.

The important idea is:

while condition:

 # logic

 # decrease/update variable

The loop terminates when the condition becomes false.

31. break STATEMENT

break is a keyword that instructs the Python virtual machine to **exit the current loop immediately**.

Example

while True:

 n = int(input("enter a number: "))

 if n == 0:

 break

 print("the entered number is :", n)

If the user enters:

0

the loop terminates.

32. continue STATEMENT

continue instructs the program to **skip the current iteration** and move to the next iteration.

Example from the notes

while True:

```
n = int(input("enter a number: "))
```

```
if n == 0:
```

```
    break
```

```
elif n == 5 or n == 3:
```

```
    continue
```

```
print("the entered number is :", n)
```

```
print("\n program executed")
```

Behaviour

If input is:

10

It is displayed.

If input is:

5

the current iteration is skipped.

If input is:

3

the current iteration is skipped.

If input is:

0

the loop exits.

The source gives exactly this break/continue example on page 20.

33. return STATEMENT

return is used inside functions.

It instructs the program to return execution from the called function back to the function call.

It can also return a value or multiple values.

Example

```
def add(a, b):
```

```
    return a + b
```

```
result = add(10, 20)
```

```
print(result)
```

Output:

30

Once return is executed, the remaining statements inside that function are not executed.

34. pass STATEMENT

pass is used when a statement is syntactically required but no action needs to be performed.

Example:

```
if True:
```

```
    pass
```

It acts as a placeholder.

35. FUNCTIONS

A **function** is an independent block of code containing logic that executes when the function is called.

A function is generally created to perform a specific task.

The source introduces functions on pages 21–22.

36. IMPORTANT FUNCTION TERMS

def

def is the keyword used to define a function.

```
def function_name():
```

```
    # logic
```

Function Name

The function name should be based on the action performed.

Example:

```
def add():
```

Here:

add → function name

Parameter

A parameter is an input variable defined in the function.

```
def add(x, y):
```

Here:

x and y → parameters

Argument

An argument is the actual value passed during the function call.

```
add(10, 20)
```

Here:

10 and 20 → arguments

37. FUNCTION SYNTAX

```
def func_name(parameters):
```

```
    # logic
```

```
    return values
```

Function call

```
var_name = function_name(arguments)
```

38. FUNCTION EXAMPLE

```
def add(x, y):
```

```
    z = x + y
```

```
    return z
```

```
result = add(10, 20)
```

```
print(result)
```

Execution

```
add(10,20)
```

↓

```
x = 10
```

```
y = 20
```

↓

```
z = 10 + 20
```

↓

```
z = 30
```

↓
return 30
↓
result = 30
Output:
30

39. FUNCTION EXECUTION

A function is executed **only when it is called**.

Example:

```
def fun1():  
    print("inside a function")
```

```
print("Before call")
```

```
fun1()  
print("After call")
```

Output:

Before call

inside a function

After call

40. ORDER OF ARGUMENTS

The order of arguments should match the order of parameters.

Example:

```
def add(x, y):  
    return x + y
```

```
print(add(10, 20))
```

Here:

x = 10

y = 20

The source specifically notes that argument order should match parameter order to avoid unexpected output.

41. USES OF FUNCTIONS

Functions provide:

1. Reusability

The same logic can be used multiple times.

2. Optimization

Functions can help reduce repeated lines of code.

3. Better organization

Specific logic can be placed inside a specific function.

4. Easy calling

A function can be called whenever its task is required.

The source lists **reusability** and **optimization/reduction of lines** as key uses of functions.

42. QUICK REVISION TABLE

Topic	Important Point
Operator	Performs an operation
Operand	Value on which operation occurs
Expression	Performs an operation
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
//	Floor division
/	Float division
%	Remainder
Comparison	Compares values
and	Logical AND
or	Logical OR
not	Reverses Boolean meaning
&, `	, ^`
=	Assignment
in	Membership
is	Identity

if	Conditional execution
elif	Multiple conditions
else	Alternate condition
for	Iteration
while	Condition-based loop
break	Exit loop
continue	Skip current iteration
return	Return from function
pass	Placeholder
range()	Generates sequence values
Function	Reusable block of code
Parameter	Function input variable
Argument	Actual value passed
def	Defines function

43. IMPORTANT EXAM POINTS

Operators

7 types:

Arithmetic

Comparison

Logical

Bitwise

Assignment

Membership

Identity

Arithmetic

+ Addition

- Subtraction

* Multiplication

** Exponentiation

// Floor division

/ Float division

% Modulo

Logical

and

or

not

Membership

in

not in

Identity

is

is not

Control Flow

Conditional

↓

if

if-else

elif

shorthand if

Looping

↓

for

while

Jumping

↓

break

continue

return

pass

Function

def → Define

parameter → Function input

argument → Actual value

return → Send value back

call → Execute function